

Lab Exercise 1: Triggering Nested Loop Join

```
7  EXPLAIN (ANALYZE, VERBOSE)
8  SELECT c.first_name, c.last_name, r.rental_date
9  FROM customer c
10 JOIN rental r ON c.customer_id = r.customer_id
11 WHERE c.customer_id = 1
```

Data Output Messages Notifications

QUERY PLAN	
text	
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.050..1.264 rows=32.00 loo...
2	Output: c.first_name, c.last_name, r.rental_date
3	Buffers: shared hit=153
4	-> Index Scan using customer_pkey on public.customer c (cost=0.28..8.29 rows=1 width=17) ...
5	Output: c.customer_id, c.store_id, c.first_name, c.last_name, c.email, c.address_id, c.active...
6	Index Cond: (c.customer_id = 1)
7	Index Searches: 1
8	Buffers: shared hit=3
9	-> Seq Scan on public.rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.021..1.2...
10	Output: r.rental_id, r.rental_date, r.inventory_id, r.customer_id, r.return_date, r.staff_id, r.last_...
11	Filter: (r.customer_id = 1)
12	Rows Removed by Filter: 16012
13	Buffers: shared hit=150
14	Planning Time: 0.180 ms
15	Execution Time: 1.296 ms

- Identify the Roles: Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?
Outer table: customer
Inner table: rental

- Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?

The customer index scan returns only 1 row, so Nested Loop is efficient. A Hash Join would have unnecessary overhead from building a hash table for such a small outer result.

- Efficiency: Look at loops in the Inner Table node. Why is it 1?

Why Inner Loop = 1: Nested Loop executes the Inner node once for each Outer row. Since the customer table returns only 1 row, the rental scan is executed once.

Lab Exercise 2: Triggering Hash Join

```
12 EXPLAIN (ANALYZE, BUFFERS)
13 SELECT f.title, l.name AS language
14 FROM film f
15 JOIN language l
16 ON f.language_id = l.language_id;
```

Data Output Messages Notifications

Showing rows: 1 to 14

	QUERY PLAN
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.028..0.513 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.010..0.267 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.013..0.014 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.008..0.009 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning:
12	Buffers: shared hit=357
13	Planning Time: 3.878 ms
14	Execution Time: 0.601 ms

```
18 SET work_mem = '64kB';
19 EXPLAIN (ANALYZE, BUFFERS)
20 SELECT f.title, l.name AS language
21 FROM film f
22 JOIN language l
23 ON f.language_id = l.language_id;
```

Data Output Messages Notifications

Showing rows: 1 to 12

	QUERY PLAN
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.044..1.077 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.018..0.379 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.017..0.018 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.007..0.009 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.278 ms
12	Execution Time: 1.221 ms

- Identify Phases: Which table is being Hashed (Built)? Which table is being Probed?

The language table is hashed during the build phase, while the film table is used in the probe phase. PostgreSQL builds a hash table using `language_id` and then probes it with each row from the film table.

- Memory Check: Look at the Hash node. What is the Memory Usage?

The Hash node shows Buckets: 1024, Batches: 1, and Memory Usage: 9kB. Therefore, the hash table requires approximately 9kB of memory. Since there is only one batch, the entire hash table fits in memory.

- Hypothesis: What happens if you run `SET work_mem = '64kB';` and run it again

After setting `work_mem` to 64kB, the Hash Join plan remains unchanged. The Hash node still shows 1024 buckets, 1 batch, and 9kB memory usage. This is because the hash table is small enough to fit within the 64kB memory limit. The execution time increased from 0.601 ms to 1.221 ms, but this difference alone does not prove that the lower `work_mem` caused the slowdown, because execution time can vary between runs.